Django DS 2025–2026

List of Suggested Projects

3ING A / 3ING B / 3ING C

This document aligns the Django DS project with current engineering practice: modern backend architecture, cloud deployment, secure software design, and AI-enabled systems.
The focus is **future-facing**: Django 6 readiness, asynchronous and event-driven integration, API styles beyond REST and GraphQL, and practical architectures for LLM applications (LLM, LLM-RAG), agent workflows (MCP, A2A), and reliable model-serving workflows.
Each topic follows the same **seven-block structure** so you connect theory (definitions, principles, trade-offs, and limits) with an evidence-driven artifact (code, experiments, benchmark notes, design validation, or a justified implementation demo). Groups select **exactly one** topic per group, subject to availability on the shared sign-up sheet.

Sign-up sheet (scan QR):

Scan to register your topic: https://docs.google.com/spreadsheets/d/1Kf9D-NaPwmkogZcblS1IuqdreTT5lS8rBPbzPYXIafU/edit?usp=sharing

Build responsibly: production quality means correctness, security, observability, and honest reporting of limitations and cost.

Contents

1 How to use this document 2

2 Part I — Django 6 readiness and core architecture 2

3 Part II — Security, privacy, and advanced defense in depth 5

4 Part III — APIs, protocols, integration patterns, MCP and A2A 8

5 Part IV — AI architecture in Django: LLM, RAG, model serving, and reliability 11

6 Part V — Frontend, admin excellence, automation, simulation, and deployment operations 14

7 Required work 19

8 Submission, dates, and organization 19

1 How to use this document

1. **Global topic numbers** (Topic 1–Topic 40) run through all parts so you can register one number on the sign-up sheet.
2. Every topic follows the same **blocks a)–g)**:
   a) **Description**;
   b) **Theoretical part**;
   c) **Practical part**;
   d) **Example plan**;
   e) **Complexity and cost analysis**;
   f) **Improvements, optimizations, and limits**;
   g) **Useful links**.
3. **Deliverables should mirror the template.** Your slides and implementation evidence should clearly cover (b)–(f); links in (g) should be curated and verifiable.
4. Prefer official documentation, standards, and peer-reviewed or well-maintained technical sources.

Topic catalogue by part

Part I: Django 6 readiness and core architecture (Topics 1–7).

Part II: Security, privacy, and advanced defense in depth (Topics 8–14).

Part III: APIs, protocols, integration patterns, MCP and A2A (Topics 15–22).

Part IV: AI architecture in Django: LLM, RAG, model serving, and reliability (Topics 23–30).

Part V: Frontend, admin excellence, automation, simulation, and deployment operations (Topics 31–40).

2 Part I — Django 6 readiness and core architecture

Topic 1. Django 6 readiness: deprecations, async direction, and migration planning

a) **Description.** Study how to keep a production Django codebase aligned with the Django 6 trajectory by mapping deprecations, dependency constraints, and rollout phases, while minimizing migration risk and service disruption, including behavior under parallel workers (migration locks, safe rolling deployments, and concurrency-friendly feature flags).

b) **Theoretical part.** Django release policy, deprecation cycles, supported Python/DB versions, and compatibility testing strategy, plus concurrency considerations during rollouts (worker models, migration locking, and transaction safety).

c) **Practical part.** Audit a Django 4.x/5.x project, produce a migration checklist, and implement at least three concrete fixes, then validate them in a multi-worker/stress scenario to ensure correctness under concurrent traffic.

d) **Example plan.** Inventory warnings; map each warning to a commit; run CI matrix; document breaking-change impact; simulate a rolling upgrade to surface lock waits, connection churn, and race conditions early.

e) **Complexity and cost analysis.** Estimate upgrade effort by module (admin, ORM, middleware, API, async workers) and include the concurrency cost of migrations (lock duration risk, rollout sequencing, and temporary operational load).

f) **Improvements, optimizations, and limits.** Use incremental upgrades and feature flags; avoid big-bang migrations; ensure migrations are safe to run while multiple workers are serving requests.

g) **Useful links.**
- Django development documentation
- Django release process
- Django download and version support

Topic 2. Async Django internals: ASGI, sync boundaries, and concurrency safety

a) **Description.** Explore Django's async stack in depth, including ASGI workers, database access limits, and the hidden cost of crossing sync/async boundaries under real concurrency pressure, with explicit attention to multithreading/threadpool behavior and sync-to-async safety.

b) **Theoretical part.** ASGI vs WSGI, thread-sensitive code, async ORM caveats, connection lifecycle, and the interaction between event loops, threadpools, and Django's request/DB connection handling.

c) **Practical part.** Build async and sync endpoints for the same DB-bound operation, then compare them under controlled load, including a thread-sensitive adapter variant and a 'blocking-guard' variant.

d) **Example plan.** Implement two variants; benchmark p50/p95 latency under increasing concurrency; inspect DB connection behavior; measure threadpool saturation and identify where blocking sneaks in. **Complexity and cost analysis.** Throughput vs latency under concurrency, plus overhead from adapters, context switching, and threadpool contention/queueing effects.

e) **Improvements, optimizations, and limits.** Keep I/O async, avoid blocking calls, isolate legacy sync blocks, and cap concurrency where threadpools or DB connections become the bottleneck.

f) **Useful links.**
- Django async support
- ASGI specification docs
- Uvicorn ASGI server

Topic 3. Advanced ORM optimization: query planning, annotations, and data access patterns

a) **Description.** Design high-performance ORM strategies for complex domains and large datasets, balancing readability with query planning, denormalization choices, and long-term maintainability, while ensuring correctness under concurrent threads/requests (no shared connection state and predictable lock behavior).

b) **Theoretical part.** Select/prefetch trade-offs, subqueries, CTEs, indexing strategy, and transaction cost, including how query evaluation and atomic blocks behave under parallel transactions.

c) **Practical part.** Optimize a slow domain workflow and show before/after SQL and timings, then validate the improvement under concurrent load to confirm reduced lock contention (not just faster single-request latency).

d) **Example plan.** Profile with Django Debug Toolbar; remove N+1; add targeted indexes; validate plans; run a parallel workload that stresses the same access pattern.

e) **Complexity and cost analysis.** Query count, IO, lock contention under parallel transactions, and memory pressure per worker.

f) **Improvements, optimizations, and limits.** Add caching and denormalization only when justified, and keep cache invalidation consistent for concurrent writers.

g) **Useful links.**

- Django DB optimization
- QuerySet API reference
- PostgreSQL EXPLAIN

Topic 4. Domain-driven design and modular monolith patterns in Django

a) **Description.** Structure large Django codebases into maintainable domain modules with explicit boundaries, clear ownership, and evolution paths that reduce coupling as teams and features grow.

b) **Theoretical part.** Bounded contexts, service layer, repository-like patterns, anti-corruption boundaries.

c) **Practical part.** Refactor one feature from a fat app into domain modules with tests.

d) **Example plan.** Define domain contracts; move orchestration to services; preserve behavior with regression tests.

e) **Complexity and cost analysis.** Developer time, regression risk, and long-term maintainability impact.

f) **Improvements, optimizations, and limits.** Balance purity with Django conventions to reduce incidental complexity.

g) **Useful links.**
- Architecture patterns with Python (DDD-oriented)
- Reusable Django apps
- Bounded Context (Martin Fowler)

Topic 5. Caching architecture in Django: local, distributed, and cache invalidation

a) **Description.** Build coherent caching strategies for mixed read/write workloads by combining cache topology, eviction design, and invalidation policies tied to business correctness constraints, with explicit handling for concurrency (thundering-herd prevention and consistency under parallel writers).

b) **Theoretical part.** Cache levels, key design, TTL strategy, dogpile effects, stale data risk, and multi-writer invalidation semantics.

c) **Practical part.** Add per-view and data-level caching with Redis and evaluate hit ratio under concurrent read/write traffic.

d) **Example plan.** Baseline metrics; define key policy; instrument cache misses; run load test with mixed concurrency; track stale reads and invalidation lag.

e) **Complexity and cost analysis.** Latency gains vs consistency risk under contention and operational overhead (cache thrash, write amplification, and monitoring).

f) **Improvements, optimizations, and limits.** Use versioned keys, targeted invalidation, and write-through selectively, plus consistency checks for simultaneous updates.

g) **Useful links.**
- Django caching framework
- Redis documentation
- django-redis-cache docs

Topic 6. Multi-tenancy in Django: schema, row-level, and hybrid strategies

a) **Description.** Compare architectural options for SaaS tenant isolation in Django across security, operability, and cost, including migration paths between row-level, schema-level, and hybrid tenancy. Designed for a 5-student group: implement tenant isolation, enforce authorization boundaries, build tenant provisioning/migration tooling, run isolation/performance tests, and complete security verification in parallel, also validating

isolation under concurrent requests from multiple tenants.

b) **Theoretical part.** Security boundaries, migration complexity, cost isolation, operational trade-offs, and race conditions during tenant switching under parallel traffic.

c) **Practical part.** Prototype one multi-tenant strategy and implement tenant-aware auth and queries, then stress it with parallel requests from multiple tenants to verify isolation.

d) **Example plan.** Build tenant middleware; add test data per tenant; verify data isolation using parallel test runners and repeated authorization checks.

e) **Complexity and cost analysis.** Per-tenant scaling, migration burden, noisy-neighbor risk, and concurrency-related isolation failures.

f) **Improvements, optimizations, and limits.** Automate tenant provisioning and schema checks, and add guardrails to prevent accidental cross-tenant query paths.

g) **Useful links.**
- django-tenants documentation
- PostgreSQL schemas
- Django multiple databases

Topic 7. Design patterns in Django: CQRS, event sourcing boundaries, and clean architecture

a) **Description.** Apply CQRS, event-sourcing boundaries, and clean architecture only where domain complexity justifies them, and define concrete signals that prevent unnecessary overengineering. Designed for a 5-student group: one workstream owns domain modeling and boundaries, one implements the command/write side, one builds the event store and replay safety, one develops the read/projection layer, and one owns integration tests plus documentation, while ensuring projections remain correct under concurrent command processing and event replay.

b) **Theoretical part.** CQRS basics, event streams vs state stores, domain events, consistency models, and idempotency requirements under concurrent consumers.

c) **Practical part.** Implement a small command/query split with outbox event publication, then run concurrent command producers while validating replay-safe projections.

d) **Example plan.** Model commands; project read views; publish domain events; run replay tests in parallel; add determinism checks for projections.

e) **Complexity and cost analysis.** Extra code paths, consistency testing needs under concurrency, observability requirements, and operational complexity.

f) **Improvements, optimizations, and limits.** Use selective adoption; avoid full event sourcing unless justified, and define clear ordering/idempotency guarantees per use case.

g) **Useful links.**
- CQRS (Martin Fowler)
- Transactional outbox pattern
- The Twelve-Factor App

3 Part II — Security, privacy, and advanced defense in depth

Topic 8. Threat modeling for Django applications (STRIDE and misuse cases)

a) **Description.** Build practical threat models for real Django systems by identifying critical assets, trust boundaries, abuse paths, and high-impact mitigations that are realistic for team capacity.

b) **Theoretical part.** Assets, attack surfaces, trust boundaries, and abuse scenarios.

c) **Practical part.** Create a threat model for one existing app and map mitigations to code/config controls.

d) **Example plan.** Data flow diagram; prioritized threat list; mitigation backlog.

e) **Complexity and cost analysis.** Security effort vs reduction of high-impact risks.

f) **Improvements, optimizations, and limits.** Revisit model each release; integrate into PR process.

g) **Useful links.**
- OWASP threat modeling
- STRIDE overview
- OWASP Cheat Sheet Series

Topic 9. Advanced authentication in Django: WebAuthn, passkeys, and MFA architecture

a) **Description.** Evaluate modern authentication flows beyond passwords, including passkeys and MFA step-up policies, with attention to usability trade-offs, recovery paths, and attack resistance.

b) **Theoretical part.** FIDO2/WebAuthn model, phishing resistance, device trust, recovery paths.

c) **Practical part.** Add MFA or passkey support to a Django auth flow.

d) **Example plan.** Baseline auth; add second factor; test lockout and recovery scenarios.

e) **Complexity and cost analysis.** User friction, support cost, and account-takeover risk reduction.

f) **Improvements, optimizations, and limits.** Progressive enrollment and risk-based auth.

g) **Useful links.**
- WebAuthn Level 2 (W3C)
- django-otp documentation
- FIDO passkeys overview

Topic 10. Secure session and token management: cookies, JWT pitfalls, and rotation policies

a) **Description.** Compare secure session-based and token-based strategies in Django systems across revocation, replay resistance, key rotation, and distributed deployment constraints.

b) **Theoretical part.** Session fixation, token theft, replay risk, expiration and refresh semantics.

c) **Practical part.** Harden a login system with secure cookie flags, rotation, and logout invalidation.

d) **Example plan.** Audit settings; add rotation logic; verify with integration tests.

e) **Complexity and cost analysis.** Security gains vs state-management complexity.

f) **Improvements, optimizations, and limits.** Prefer HttpOnly/SameSite cookies when possible.

g) **Useful links.**
- Django sessions
- Django security topics
- RFC 7519 (JWT)

Topic 11. Advanced authorization: RBAC, ABAC, and policy engines in Django

a) **Description.** Design scalable authorization beyond simple role checks by combining RBAC and ABAC policies, centralized decision points, and auditable enforcement at service boundaries.

b) **Theoretical part.** RBAC vs ABAC, policy composition, context-aware decisions.

c) **Practical part.** Implement object-level policy checks with explainable deny reasons.

d) **Example plan.** Define roles and attributes; enforce policy at service/API layers.

e) **Complexity and cost analysis.** Decision latency, policy sprawl, and auditability.

f) **Improvements, optimizations, and limits.** Externalize high-variance policy logic.

g) **Useful links.**
- django-guardian docs
- Open Policy Agent docs
- NIST RBAC project

Topic 12. Secrets management and key lifecycle in Django deployments

a) **Description.** Manage secrets securely across environments and CI/CD using lifecycle controls for generation, rotation, scoped access, and incident-ready revocation procedures.

b) **Theoretical part.** Secret sprawl, envelope encryption, rotation windows, and blast radius.

c) **Practical part.** Replace static secrets with managed vault-backed retrieval.

d) **Example plan.** Inventory secrets; move to vault; enforce rotation and access logging.

e) **Complexity and cost analysis.** Operational overhead vs leakage risk reduction.

f) **Improvements, optimizations, and limits.** Use short-lived credentials where possible.

g) **Useful links.**
- 12-factor config
- HashiCorp Vault docs
- GitHub secret scanning

Topic 13. Data privacy and compliance-by-design in Django systems

a) **Description.** Engineer privacy controls into architecture from the start through data minimization, retention boundaries, consent-aware processing, and verifiable compliance evidence.

b) **Theoretical part.** Data minimization, lawful basis, retention, right-to-erasure, audit trails.

c) **Practical part.** Implement retention + anonymization workflow for selected models.

d) **Example plan.** Build data map; mark sensitive fields; add deletion/anonymization jobs.

e) **Complexity and cost analysis.** Storage and legal risk costs vs implementation effort.

f) **Improvements, optimizations, and limits.** Add privacy checks to schema review and CI.

g) **Useful links.**
- GDPR overview
- Django security recommendations
- UK GDPR guidance (ICO)

Topic 14. Software supply-chain security for Django projects (SCA, SBOM, signed builds)

a) **Description.** Secure dependencies, build artifacts, and release pipelines with continuous vulnerability analysis, provenance checks, and tamper-resistant delivery controls.

b) **Theoretical part.** Vulnerability databases, provenance, signed attestations, and update policies.

c) **Practical part.** Add dependency scanning and produce an SBOM for a Django project.

d) **Example plan.** Configure Dependabot/pip-audit; generate SBOM; triage CVEs.

e) **Complexity and cost analysis.** Alert volume, false positives, and patch latency.

f) **Improvements, optimizations, and limits.** Risk-based patching and exception governance.

g) **Useful links.**
- pip-audit documentation
- CycloneDX SBOM standard
- SLSA framework

4 Part III — APIs, protocols, integration patterns, MCP and A2A

Topic 15. gRPC APIs with Django: schema-first contracts and performance trade-offs

a) **Description.** Build binary contract-based APIs for internal platform communication using schema-first governance, explicit backward-compatibility rules, and measurable latency/error objectives. Designed for a 5-student group: define proto contracts, implement the Django gRPC service, generate/validate clients, benchmark performance, and deliver observability and error-handling evidence.

b) **Theoretical part.** Proto contracts, backward compatibility, streaming, and language interoperability.

c) **Practical part.** Implement one gRPC service backed by Django domain logic.

d) **Example plan.** Define proto; generate stubs; benchmark against JSON HTTP endpoint.

e) **Complexity and cost analysis.** Throughput, serialization overhead, and operability complexity.

f) **Improvements, optimizations, and limits.** Use gRPC for service-to-service, keep public APIs simpler.

g) **Useful links.**
- gRPC documentation
- Protocol Buffers
- django-grpc-framework (GitHub)

Topic 16. AsyncAPI and event contracts for Django microservices

a) **Description.** Design event-driven contracts with explicit schema governance, ownership rules, and compatibility testing so producers and consumers can evolve independently.

b) **Theoretical part.** Publish/subscribe semantics, topic taxonomy, idempotency, and ordering.

c) **Practical part.** Produce AsyncAPI docs and implement one event producer + consumer.

d) **Example plan.** Define events; implement outbox dispatch; validate schema in CI.

e) **Complexity and cost analysis.** Consumer complexity, replay handling, and observ-

ability effort.

f) **Improvements, optimizations, and limits.** Enforce schema compatibility checks.

g) **Useful links.**
- AsyncAPI docs
- Apache Kafka documentation
- RabbitMQ documentation

Topic 17. WebSockets, SSE, and realtime channels in Django systems

a) **Description.** Compare realtime delivery patterns and reliability guarantees across WebSockets, SSE, and broker-assisted fan-out, including failure recovery and client reconnection behavior, with per-connection concurrency safety (no shared mutable state across clients; isolate sync code from threadpools).

b) **Theoretical part.** Duplex vs server-push models, backpressure, fan-out, and reconnection under worker concurrency.

c) **Practical part.** Implement one realtime feature with measurable SLIs under multiple simultaneous clients and reconnect storms.

d) **Example plan.** Build notifications stream; test disconnect/retry, rate limits, and reconnect storms while confirming ordering/duplication expectations.

e) **Complexity and cost analysis.** Connection footprint, broker load, fan-out cost, and concurrency-related resource usage (threads/event loops).

f) **Improvements, optimizations, and limits.** Use adaptive batching and connection lifecycle policies, and define explicit behavior when backpressure builds up.

g) **Useful links.**
- Django Channels docs
- MDN: Server-Sent Events
- WebSocket protocol (RFC 6455)

Topic 18. Schema evolution beyond REST/GraphQL: protobuf contracts, gRPC-Web gateways, and backward compatibility

a) **Description.** Engineer long-lived API contracts with strict compatibility controls by formalizing schema evolution policies, deprecation timelines, and automated breaking-change detection. Designed for a 5-student group: define versioning/deprecation rules, implement the gateway translation layer, build a compatibility test suite for old/new consumers, run rollout safety benchmarks, and document migration playbooks.

b) **Theoretical part.** Compatibility modes, field evolution rules, gateway patterns, and consumer-driven contracts.

c) **Practical part.** Build a protobuf-first API with an HTTP gateway and compatibility tests.

d) **Example plan.** Version proto schema; generate clients; test old and new consumers against an updated server.

e) **Complexity and cost analysis.** Contract governance overhead versus reduced breaking-change incidents.

f) **Improvements, optimizations, and limits.** Use linted API design rules and explicit deprecation windows.

g) **Useful links.**
- Protocol Buffers proto3 guide
- gRPC core concepts

- Buf documentation (protobuf lint and breaking checks)

Topic 19. MCP for Django applications: tool interfaces for LLM-driven workflows

a) **Description.** Use Model Context Protocol (MCP) to expose Django capabilities as safe tools with strict input validation, permission scoping, and observability for every tool invocation.

b) **Theoretical part.** MCP concepts, tool schemas, context boundaries, and permission models.

c) **Practical part.** Prototype one MCP-compatible interface backed by Django services.

d) **Example plan.** Define tool contracts; add access control; test abuse and prompt-injection paths.

e) **Complexity and cost analysis.** Tool orchestration latency, auditing overhead, and reliability risk.

f) **Improvements, optimizations, and limits.** Minimize tool side effects and require strict validation.

g) **Useful links.**
- Model Context Protocol (MCP) introduction
- MCP specification
- MCP servers (GitHub)

Topic 20. A2A (Agent-to-Agent) workflows with Django orchestration

a) **Description.** Analyze multi-agent exchange protocols and governance in enterprise systems, focusing on bounded autonomy, negotiation contracts, timeout handling, and accountability.

b) **Theoretical part.** Agent roles, message contracts, coordination failure modes, and auditability.

c) **Practical part.** Simulate two agents coordinating tasks via Django-managed workflow state.

d) **Example plan.** Define task graph; add retries/timeouts; inspect failure-recovery traces.

e) **Complexity and cost analysis.** Extra orchestration calls, non-determinism cost, and debugging effort.

f) **Improvements, optimizations, and limits.** Use strict state machines and bounded execution budgets.

g) **Useful links.**
- Agent2Agent (A2A) protocol documentation
- ActivityStreams 2.0 core (W3C)
- Temporal workflow platform

Topic 21. API gateway and service mesh patterns for Django ecosystems

a) **Description.** Centralize policy, observability, and traffic control for distributed Django services through gateway and mesh patterns that improve security posture without hiding system complexity. Designed for a 5-student group: configure north-south gateway policies, implement retries/circuit breakers and mTLS assumptions, integrate tracing/metrics, validate rate limiting, and produce a threat-model-based runbook.

b) **Theoretical part.** North-south vs east-west traffic, retries, circuit breakers, and mTLS.

c) **Practical part.** Deploy a gateway with authentication and rate limiting for two services.

d) **Example plan.** Define routes; enforce auth; collect latency and error metrics.

e) **Complexity and cost analysis.** Added hops, operational complexity, and policy consistency.

f) **Improvements, optimizations, and limits.** Keep gateway concerns focused; avoid business logic there.

g) **Useful links.**
- Kubernetes Gateway API
- Istio documentation
- Envoy documentation

Topic 22. Event-driven integration at scale: outbox, CDC, and idempotent consumers

a) **Description.** Build reliable asynchronous integration for high-scale Django systems with outbox and CDC patterns, idempotent consumers, and replay-safe operational workflows. Designed for a 5-student group: implement the outbox producer, enforce event schema governance, build an idempotent consumer with replay safety (including parallel consumers), add operational controls (DLQ/retries), and validate end-to-end scenarios under concurrent processing.

b) **Theoretical part.** At-least-once delivery, deduplication under concurrency, exactly-once myths, and ordering guarantees when multiple consumers process events in parallel.

c) **Practical part.** Implement outbox pattern and idempotent consumers with replay tests that simulate concurrent deliveries and duplicate processing.

d) **Example plan.** Add outbox table; publish events; test crash-recovery scenarios.

e) **Complexity and cost analysis.** Write amplification, broker storage, and replay processing cost.

f) **Improvements, optimizations, and limits.** Use compacted topics and retention by domain need.

g) **Useful links.**
- Transactional outbox pattern
- Debezium CDC documentation
- Apache Kafka docs

5 Part IV — AI architecture in Django: LLM, RAG, model serving, and reliability

Topic 23. LLM application architecture in Django: orchestration, prompts, and control planes

a) **Description.** Architect robust Django backends for LLM-enabled user workflows with prompt governance, context budgeting, fallback strategies, and explicit boundaries between deterministic and generative logic. Designed for a 5-student group: own the prompt/control plane, integrate tool calls to Django services, implement guardrails and structured outputs, build evaluation and cost instrumentation, and validate failure-handling end-to-end.

b) **Theoretical part.** Prompt lifecycle, context windows, tool calls, and hallucination controls.

c) **Practical part.** Build a small production-style LLM endpoint with safety checks.

d) **Example plan.** Implement guardrails, structured outputs, and fallback behavior.

e) **Complexity and cost analysis.** Token cost, p95 latency, and retrial overhead.

f) **Improvements, optimizations, and limits.** Use caching, truncation policies, and

response validation.

g) **Useful links.**
- OpenAI API docs
- Anthropic documentation
- Google Gemini API docs

Topic 24. Graph-RAG in Django: entity extraction, multi-hop reasoning, and provenance tracing

a) **Description.** Design Graph-RAG pipelines for questions that require relations across multiple documents, emphasizing entity quality, multi-hop retrieval, and provenance traceability for trust. Designed for a 5-student group: own entity/relation extraction, graph indexing, multi-hop retrieval and ranking, provenance/citation confidence scoring, and an evaluation set that measures faithfulness and coverage.

b) **Theoretical part.** Entity and relation extraction, graph traversal, path scoring, and provenance confidence.

c) **Practical part.** Build a small knowledge graph from domain documents and answer multi-hop queries.

d) **Example plan.** Extract entities; build graph index; compare graph-assisted answers versus vanilla vector RAG.

e) **Complexity and cost analysis.** Graph construction cost, traversal latency, and storage overhead.

f) **Improvements, optimizations, and limits.** Use incremental graph updates and confidence-based abstention.

g) **Useful links.**
- Neo4j documentation
- NetworkX documentation
- LlamaIndex docs

Topic 25. Model serving for Django: online inference, batching, and model lifecycle

a) **Description.** Integrate model-serving backends with Django workflows across synchronous and asynchronous paths, including autoscaling behavior, cold-start impact, and version lifecycle strategy, while ensuring batching/queueing logic is concurrency-safe across workers and avoids shared mutable request state. Designed for a 5-student group: implement serving integration, design batching/queueing, manage model version lifecycle and fallbacks, run performance/throughput benchmarks, and validate reliability plus drift monitoring.

b) **Theoretical part.** Serving architecture, cold starts, autoscaling, drift considerations, and concurrency/throughput constraints (queue depth, worker boundaries, and batching limits).

c) **Practical part.** Connect Django to one serving framework and benchmark end-to-end inference under concurrent traffic and controlled load bursts.

d) **Example plan.** Deploy model endpoint; add async queue; compare sync vs async inference under concurrent load; measure queue delay and cold-start impact.

e) **Complexity and cost analysis.** GPU/CPU cost, queue delay, and throughput constraints.

f) **Improvements, optimizations, and limits.** Dynamic batching and adaptive routing.

g) **Useful links.**
- Ray Serve docs

- KServe docs
- BentoML documentation

Topic 26. Prompt and model evaluation pipelines: reliability engineering for LLM features

a) **Description.** Treat LLM behavior as a testable, monitored software component with offline/online evaluation loops, regression thresholds, and release gates tied to risk levels. Designed for a 5-student group: build the evaluation dataset and labeling rubric, implement offline metrics and regression gates, connect online guard metrics, create reporting dashboards, and test release workflows.

b) **Theoretical part.** Offline evaluation, online guard metrics, regression detection, and failure taxonomies.

c) **Practical part.** Build an eval harness with fixed test set and release gates.

d) **Example plan.** Define metrics; run nightly eval; compare model or prompt versions.

e) **Complexity and cost analysis.** Eval runtime and labeling effort vs incident prevention value.

f) **Improvements, optimizations, and limits.** Use risk-tiered eval suites and canary rollouts.

g) **Useful links.**
- LangSmith documentation
- OpenAI Evals (GitHub)
- MLflow docs

Topic 27. AI agent safety in Django products: tool permissions, sandboxing, and audit logs

a) **Description.** Prevent unsafe tool actions and privilege escalation in agent workflows by combining capability-based permissions, isolation boundaries, and high-confidence audit trails.

b) **Theoretical part.** Capability-based security, execution isolation, and policy enforcement points.

c) **Practical part.** Implement permission-scoped tool execution with full audit trail.

d) **Example plan.** Add allowlists; block dangerous commands; log every tool invocation.

e) **Complexity and cost analysis.** Added security checks vs UX/latency impact.

f) **Improvements, optimizations, and limits.** Human approval for high-risk actions.

g) **Useful links.**
- OWASP Top 10 for LLM applications
- NIST AI RMF 1.0
- Anthropic tool use docs

Topic 28. Hybrid retrieval stacks: BM25 + dense vectors + reranking in Django backends

a) **Description.** Blend lexical and semantic retrieval for better robustness by designing query routing, score fusion, and reranking policies that adapt to question intent and corpus drift.

b) **Theoretical part.** Precision/recall balance, score fusion, and reranking trade-offs.

c) **Practical part.** Implement hybrid retrieval and compare against pure dense retrieval.

d) **Example plan.** Build corpus; add BM25 baseline; fuse top-k with reranker.

e) **Complexity and cost analysis.** Extra retrieval stages vs measurable quality gains.

f) **Improvements, optimizations, and limits.** Adaptive query routing by query type.

g) **Useful links.**
- Hybrid search primer (Pinecone)
- Elasticsearch query DSL
- Sentence-Transformers docs

Topic 29. Vector infrastructure choices for Django AI apps: FAISS, pgvector, and managed stores

a) **Description.** Compare local and managed vector infrastructure under realistic workload assumptions, including ANN recall trade-offs, operational burden, and total cost over growth phases.

b) **Theoretical part.** Index families, ANN accuracy trade-offs, and operational ownership.

c) **Practical part.** Prototype two backends and compare latency/recall/cost.

d) **Example plan.** Build same corpus; run identical query set; report p95 and recall@k.

e) **Complexity and cost analysis.** Build cost, storage footprint, and scaling breakpoints.

f) **Improvements, optimizations, and limits.** Use tiered storage and background compaction.

g) **Useful links.**
- FAISS documentation
- pgvector (GitHub)
- Qdrant documentation

Topic 30. Multimodal AI with Django: document, image, and structured data pipelines

a) **Description.** Integrate multimodal AI features in full-stack Django products with robust ingestion pipelines, modality-aware routing, and confidence-driven fallback behavior.

b) **Theoretical part.** Modality routing, extraction reliability, and storage/retention concerns.

c) **Practical part.** Build a small ingestion-to-answer pipeline for documents and images.

d) **Example plan.** Parse data; classify modality; route to correct model chain.

e) **Complexity and cost analysis.** Parsing overhead, model latency, and storage growth.

f) **Improvements, optimizations, and limits.** Add confidence thresholds and fallback paths.

g) **Useful links.**
- Transformers docs
- LlamaIndex docs
- Pydantic documentation

6 Part V — Frontend, admin excellence, automation, simulation, and deployment operations

Topic 31. Advanced frontend architecture for Django: micro-frontends, BFF, and edge rendering

a) **Description.** Compare micro-frontend and backend-for-frontend (BFF) architectures for large product teams, analyzing coupling, independent deployability, and end-to-end performance implications. Designed for a 5-student group: one team owns micro-frontend composition, one owns BFF/edge endpoints, one runs performance/SEO experiments, one validates UX/accessibility policies, and one drives deployment automation.

b) **Theoretical part.** Team topology alignment, composition models, edge rendering, and cache invalidation.

c) **Practical part.** Implement one feature using a BFF layer and modular frontend slices.

d) **Example plan.** Build BFF endpoint; split UI modules; validate latency and deployment independence.

e) **Complexity and cost analysis.** Added architecture complexity versus team autonomy and release velocity.

f) **Improvements, optimizations, and limits.** Use shared contracts and strict versioned frontend APIs.

g) **Useful links.**
- Micro-frontends guide
- Next.js rendering patterns
- MDN web performance

Topic 32. Django admin as a governed operations platform: policy enforcement, immutable logs, and approval workflows

a) **Description.** Transform Django admin into a controlled operations surface for sensitive business actions with policy gates, separation of duties, and tamper-evident traceability.

b) **Theoretical part.** Separation of duties, dual control, append-only audit logs, and tamper evidence.

c) **Practical part.** Implement approval-gated admin actions with immutable history for critical changes.

d) **Example plan.** Add action policy checks; require second approver; record a cryptographic hash chain for logs.

e) **Complexity and cost analysis.** Extra operator steps versus significant reduction in privileged misuse risk.

f) **Improvements, optimizations, and limits.** Scope governance to high-risk actions to avoid process overload.

g) **Useful links.**
- Django admin reference
- OWASP logging cheat sheet
- NIST SP 800-53 overview

Topic 33. Progressive delivery for Django: canary, blue-green, and shadow traffic strategies

a) **Description.** Apply progressive delivery to reduce release risk in high-availability Django systems using canary, blue-green, and shadow traffic stages governed by objective health signals. Designed for a 5-student group: implement traffic shifting, define health/SLO gates, validate rollback safety, run shadow/canary comparisons, and write operational runbooks with evidence.

b) **Theoretical part.** Deployment topologies, traffic shifting, rollback semantics, and blast-radius control.

c) **Practical part.** Implement a staged rollout policy with automated health gates.

d) **Example plan.** Deploy canary; mirror shadow traffic; compare error budgets before full rollout.

e) **Complexity and cost analysis.** Additional infrastructure and monitoring cost versus incident-prevention gains.

f) **Improvements, optimizations, and limits.** Use automated rollback and SLO-aware

promotion checks.

g) **Useful links.**
- Argo Rollouts documentation
- Canary release (Martin Fowler)
- Google SRE workbook: canarying releases

Topic 34. Deep observability for Django and AI pipelines: OpenTelemetry, eBPF signals, and SLO burn-rate alerting

a) **Description.** Build high-fidelity observability for distributed Django and model-serving components by correlating traces, metrics, and logs into SLO-driven operational decision-making. Designed for a 5-student group: instrument traces, define RED/USE metrics and SLOs, design log correlation with PII-safe redaction, implement alerting playbooks, and perform incident drills.

b) **Theoretical part.** Correlated traces/metrics/logs, kernel-level telemetry, and multi-window burn-rate alerts.

c) **Practical part.** Instrument service and worker paths; define SLOs; implement actionable alerting.

d) **Example plan.** Add trace propagation; collect RED metrics; configure fast and slow burn-rate alert rules.

e) **Complexity and cost analysis.** Cardinality and storage cost versus mean-time-to-detect and diagnose improvements.

f) **Improvements, optimizations, and limits.** Tail-based sampling and PII-safe telemetry pipelines.

g) **Useful links.**
- OpenTelemetry documentation
- What is eBPF
- Google SRE workbook: alerting on SLOs

Topic 35. Stateful AI agent execution in Django: resumable workflows, sagas, and idempotent tool calls

a) **Description.** Build a stateful agent execution pipeline in Django that can resume after crashes while preserving correctness through saga-style orchestration and idempotent tool calls, including protection against concurrent resumptions that could duplicate side effects. Designed for a 5-student group: one workstream owns the execution journal and resume logic, one implements idempotency keys and side-effect guards, one builds the saga/transition model, one implements replay-safe tool wrappers, and one evaluates failure-recovery evidence.

b) **Theoretical part.** State machines, saga orchestration vs orchestration graphs, idempotency guarantees, deduplication windows, and consistency trade-offs under retries, including journal locking/optimistic concurrency.

c) **Practical part.** Implement an agent workflow with persistent execution records, simulate mid-run failures and concurrent retries/resumptions, and prove that re-running does not duplicate side effects.

d) **Example plan.** Add a persistent "execution id" and tool-call request ids; interrupt the run; resume from the last safe state; validate that external effects happen exactly once (or are safely idempotent).

e) **Complexity and cost analysis.** Journal storage growth, concurrency/locking cost, retry overhead, and the operational cost of maintaining idempotency stores.

f) **Improvements, optimizations, and limits.** Use bounded retries, selective state snapshots, and a clear failure taxonomy (retryable vs non-retryable).

g) **Useful links.**
- Saga pattern (microservices.io)
- LangGraph documentation (stateful agent graphs)
- Patterns of Distributed Systems (reference)

Topic 36. n8n as an external workflow engine for Django agents: webhook orchestration, retries, and secure integrations

a) **Description.** Integrate Django-based agent tools with n8n workflows so that complex multi-step actions run out-of-process, while Django remains responsible for authorization, idempotency, and security, including correct behavior under concurrent webhook deliveries and retries. Designed for a 5-student group: one team builds secure webhook endpoints, one configures the n8n workflow and credentials handling, one implements Django-to-n8n correlation and retries, one validates idempotency under duplicate webhook deliveries, and one produces a security/performance report.

b) **Theoretical part.** Workflow decoupling, at-least-once delivery implications, webhook signature validation, correlation IDs, and end-to-end retry/backoff strategies, with idempotency store consistency under parallel callback handling.

c) **Practical part.** Implement a Django "agent action" endpoint that triggers an n8n workflow via webhook; accept n8n callbacks; ensure replay-safe state transitions; and verify correct behavior under parallel callback arrivals.

d) **Example plan.** Trigger an n8n workflow; simulate retry deliveries from n8n; verify that Django processes callbacks exactly once using a stored delivery id.

e) **Complexity and cost analysis.** Operational overhead of a second orchestrator, webhook delivery latency, and the cost of signature verification and idempotency storage.

f) **Improvements, optimizations, and limits.** Add circuit breakers for n8n outages, enforce strict timeouts, and document safe fallback behavior.

g) **Useful links.**
- n8n documentation
- n8n Webhook node documentation
- Django security documentation

Topic 37. django-axes in security-by-design: adaptive lockout middleware and brute-force defense for agent-facing APIs

a) **Description.** Use 'django-axes'-style protection as a first-class cybersecurity control for agent-facing authentication endpoints and sensitive tool-execution routes, with correct lockout behavior under parallel auth attempts. Designed for a 5-student group: one team integrates and tunes axes settings, one designs endpoint scope (which routes get protected), one builds automated lockout tests, one evaluates false-positive trade-offs, and one integrates alerts into operational evidence.

b) **Theoretical part.** Brute-force attack surface, lockout thresholds, adaptive rate limiting, and safe recovery paths (password reset, admin override, and audit trails), including atomicity for lockout counters under concurrency.

c) **Practical part.** Apply axes middleware to authentication and API routes used by agents, then verify behavior under repeated failed attempts and legitimate high-frequency users, including concurrent bursts of failures.

d) **Example plan.** Build a small test harness that sends repeated auth failures; confirm lockout behavior, verify response consistency, and validate that agent-trigger endpoints

are protected.

e) **Complexity and cost analysis.** Increased latency and operational tuning cost, lockout store overhead, and risk of user friction if thresholds are misconfigured.

f) **Improvements, optimizations, and limits.** Use per-route policy, monitor lockout rates, and implement safe administrative recovery.

g) **Useful links.**
- django-axes (GitHub repository)
- OWASP Authentication Cheat Sheet
- Django security documentation

Topic 38. Middleware-level cybersecurity for AI tool execution in Django: policy enforcement, SSRF/command injection prevention, and auditing

a) **Description.** Implement middleware that constrains and validates AI tool calls at request-time, preventing dangerous outbound calls and unsafe command patterns while keeping auditable traces for incident response, with thread-safe policy evaluation and correct audit writes under concurrent requests. Designed for a 5-student group: one team designs a capability/allowlist model, one implements SSRF protections for outbound destinations, one implements command/injection defenses for tool payloads, one builds middleware test cases (including attack simulations), and one manages tamper-evident auditing.

b) **Theoretical part.** Middleware placement in Django request lifecycle, capability-based security for tools, SSRF and injection threat classes, and audit integrity for forensics, including parallel request processing considerations.

c) **Practical part.** Add a protected "tool-call" endpoint where middleware validates payloads (destinations, parameters), enforces authorization, and writes tamper-evident audit records; test it with parallel tool-call requests including malicious payloads.

d) **Example plan.** Create an allowlist of safe domains/operations; send malicious requests (SSRF and injection patterns); verify blocked actions and preserved audit evidence.

e) **Complexity and cost analysis.** Middleware overhead per request, additional policy evaluation cost, and storage cost for audit trails.

f) **Improvements, optimizations, and limits.** Prefer safe defaults (deny-by-default), keep validation deterministic, and document what is intentionally unsupported.

g) **Useful links.**
- OWASP SSRF Prevention Cheat Sheet
- OWASP OS Command Injection Defense Cheat Sheet
- OWASP Logging Cheat Sheet

Topic 39. Forensic replay for agent incidents in Django: immutable event logs, redaction, and chain-of-custody

a) **Description.** Create a forensic pipeline that records agent tool execution events as immutable logs, applies privacy-safe redaction, and enables controlled forensic replay without repeating side effects. Designed for a 5-student group: one team builds immutable logging and hash chaining, one team implements deterministic redaction rules for prompts/data, one team implements a safe replay mode, one team validates integrity and reproducibility, and one team writes an incident response playbook.

b) **Theoretical part.** Audit-event integrity, chain-of-hashes for tamper evidence, redaction vs usefulness trade-offs, and incident response workflows for agent systems.

c) **Practical part.** Implement event logging for tool calls, store redacted snapshots, and

create a "replay analyzer" that reconstructs the sequence for investigation.

d) **Example plan.** Simulate a malicious or faulty tool execution; capture immutable events; verify chain integrity; run replay in safe mode; demonstrate investigation reproducibility.

e) **Complexity and cost analysis.** Event storage volume, indexing overhead for investigation queries, and privacy risk management cost.

f) **Improvements, optimizations, and limits.** Use retention policies, privacy budgets, and scoped replay permissions.

g) **Useful links.**
- NIST SP 800-92: Guide to Computer Security Log Management
- OWASP Logging Cheat Sheet
- Patterns of Distributed Systems (reference)

Topic 40. Chaos engineering and resilience simulation for Django + AI production paths

a) **Description.** Validate system resilience by controlled fault injection and simulation-driven testing, defining steady-state hypotheses and measurable recovery expectations before production incidents. Designed for a 5-student group: define steady-state hypotheses, implement fault injection modules, measure recovery and data integrity, automate experiments safely, and produce a resilience report with lessons learned.

b) **Theoretical part.** Steady-state hypotheses, failure domains, blast radius, and rollback safety.

c) **Practical part.** Execute controlled chaos experiments on queues, caches, and model endpoints.

d) **Example plan.** Define a hypothesis; inject one failure at a time; measure recovery time and user impact.

e) **Complexity and cost analysis.** Experiment overhead versus risk reduction from validated recovery behavior.

f) **Improvements, optimizations, and limits.** Start in staging and progressively expand controlled production experiments.

g) **Useful links.**
- Principles of Chaos Engineering
- LitmusChaos documentation
- Resilience4j documentation

7 Required work

The project has **two mandatory components**, aligned with blocks **(b)** and **(c)** of your chosen topic; blocks **(d)**–**(f)** must be visible in slides and practical evidence.

I. **Theoretical part (block (b)).** Use reputable sources (official docs, standards, books, peer-reviewed papers, high-quality engineering references). Include explicit **complexity/cost** and **improvements/limits**.

II. **Practical part (block (c)).** Implement a meaningful prototype, benchmark, architecture experiment, or justified technical demo with reproducible evidence.

8 Submission, dates, and organization

Step 1: Form a group (recommended size: 4 students; topics well suited for 5 students: Topic 6, Topic 7, Topic 15, Topic 18, Topic 21, Topic 22, Topic 23, Topic 24, Topic 25, Topic 26, Topic 31, Topic 33, Topic 34, Topic 35, Topic 36, Topic 37, Topic 38, Topic 39, Topic 40).

Step 2: Claim a topic.

a) Email the instructor with topic number, section (**3ING A**, **3ING B**, or **3ING C**), and full member names.

b) Register the same choice on the shared Google Sheet: https://docs.google.com/spreadsheets/d/1Kf9D-NaPwmkogZcblS1IuqdreTT5lS8rBPbzPYXIafU/edit?usp=sharing.

c) Topics are assigned **first come, first served**; each topic number is assigned to **one group only** across **3ING A/B/C combined**.

Step 3: Deposit on Moodle by uploading each artifact in its own Moodle deposit space: slides (PDF or PPT), source code (repo link or ZIP), and screenshot/demo-video evidence (tests/runs).

Step 4: Deadline (email): Sunday, 26 April 2026, 23:59 (last Sunday of April).

Step 5: Oral presentations: Monday, 27 April 2026 and **Tuesday, 28 April 2026**.

No additional time will be granted after the deadline.